

Azure-Status

Overview

An automated, serverless DevOps pipeline designed to track live Azure resource consumption, verify web availability and generate visual health reports delivered straight to Slack.

By combining cloud infrastructure APIs, headless browser automation, and multi-platform CI/CD scheduling, this project eliminates the visibility gap in daily infrastructure monitoring.

Technologies involved:

- PowerShell
- YAML Scripting
- GitHub / GitHub Actions
- GitLab / GitLab CI/CD Pipeline

Contents

Overview	1
PowerShell Script	3
GitHub Action.....	6
GitLab Schedule	7
Slack Message	8
Conclusion.....	9

PowerShell Script

The script first gets the Month-to-Date cost for the resources used in Azure through a REST API call. The information is then grouped by the instance/resource name to account for duplicate records and each resources cost are summed up. A custom object is then created to hold only the resource name and Month-to-Date costs that will be displayed as a table in the Slack message.

```
# Get Billing
#-----
az rest --method get --uri "https://management.azure.com/subscriptions/$SubscriptionID/providers/Microsoft.Consumption/usageDetails?api-version=2021-10-01" --output json > raw_billing.json
$BillingData = Get-Content -Raw -Path ".\raw_billing.json" | ConvertFrom-Json
$GroupedData = $BillingData.value | Group-Object -Property { $_.properties.instanceName }
$ProcessedResources = foreach ($Bucket in $GroupedData) {
    $Sum = 0
    foreach ($Record in $Bucket.Group) {
        $Sum += $Record.properties.paygCostInUSD
    }
}
[PSCustomObject]@(
    [PSCustomObject]@{
        Name = $Bucket.Name.Split('/')[0]
        Cost = [Math]::Round($Sum, 5)
    }
)
#-----
```

It then performs the same process for my live Azure Static Web App through a separate REST API call. The target URL and custom domains are extracted from the response and used in curl requests to verify their status responses. A custom object is then created to store the URLs (target URL and custom domains) along with the corresponding status codes returned by curl.

```
# Get Static Web App
#-----
$ActiveAssets = az resource list --output json | ConvertFrom-Json
$WebAssets = $ActiveAssets | where-object { $_.name -like "myresource*" }
$WebDetails = az rest --method get --uri "https://management.azure.com/subscriptions/$SubscriptionID/resourceGroups/$($WebAssets.resourceGroup)/providers/Microsoft.Web/staticSites/$($WebAssets.name)?api-version=2023-05-01" | ConvertFrom-Json
$WebURI = $WebDetails.properties.defaultHostName
$CustomDomains = $WebDetails.properties.customDomains
$UriArray = @(
    "https://$WebURI"
)
foreach ($Domain in $CustomDomains) {
    $UriArray += "https://$Domain"
}
$UriTable = foreach ($Uri in $UriArray) {
    [PSCustomObject]@{
        "Target URL" = $Uri
        "Status Code" = (Invoke-WebRequest -Uri $Uri -Method Head -UseBasicParsing).StatusCode
    }
}
#-----
```

```

# Screenshot WebApp
=====
if (!(Test-Path "docs")) { New-Item -ItemType Directory -Path "docs" }

$SlackImageBlocks = @()

if ($env:GITHUB_ACTIONS -eq "true") {
    Write-Host "Running in GitHub Actions. Bootstrapping Headless Browser..."

    pip install shot-scraper playwright pillow --quiet
    playwright install chromium --with-deps

    $UtcTime = [DateTime]::UtcNow
    $SgtZone = [System.TimeZoneInfo]::FindSystemTimeZoneById("Singapore Standard Time")
    $SgtTime = [System.TimeZoneInfo]::ConvertTimeBySystemTimeZoneId($UtcTime, "Singapore Standard Time")
    $TimestampText = "Captured: $($SgtTime.ToString('yyyy-MM-dd HH:mm')) SGT"

    for ($i = 0; $i -lt $UrlArray.Count; $i++) {
        $CacheBuster = Get-Date -Format "yyyyMMddHHmmss"
        $TargetUrl = $UrlArray[$i]
        $CleanFileName = $TargetUrl -replace "https://", "" -replace "/", "-"
        $FileName = "screenshot-$CleanFileName.png"
        $ScreenshotPath = "docs/$FileName"

        Write-Host "Capturing: $TargetUrl -> $ScreenshotPath"
        shot-scraper $TargetUrl -o $ScreenshotPath --width 1280 --height 800 --wait 5000

        if (Test-Path $ScreenshotPath) {
            Write-Host "Watermarking timestamp onto $FileName..."
            python -c @"
from PIL import Image, ImageDraw, ImageFont
import os

img_path = '$ScreenshotPath'
if os.path.exists(img_path):
    img = Image.open(img_path).convert('RGBA')
    txt = Image.new('RGBA', img.size, (255,255,255,0))

    d = ImageDraw.Draw(txt)

    d.rectangle([img.size[0] - 340, img.size[1] - 60, img.size[0] - 10, img.size[1] - 10], fill=(0,0,0,160))
    d.text((img.size[0] - 330, img.size[1] - 52), '$TargetUrl', fill=(255,255,255,255))
    d.text((img.size[0] - 330, img.size[1] - 30), '$TimestampText', fill=(255,255,255,255))

    final_img = Image.alpha_composite(img, txt).convert('RGB')
    final_img.save(img_path, 'PNG')
"@

            $GitHubRawUrl = "https://raw.githubusercontent.com/Lacketronik/Azure-Status/main/docs/$FileName?v=$CacheBuster"

            $SlackImageBlocks += @(
                type = 'image'
                title = @{ type = 'plain_text'; text = "Visual Check: $CleanFileName" }
                image_url = $GitHubRawUrl
                alt_text = $CleanFileName
            )

            if ($i -lt ($UrlArray.Count - 1)) { $SlackImageBlocks += @{ type = 'divider' } }
        }
    }
} else {
    Write-Host "Local execution detected. Skipping headless screenshots to avoid environment errors."
}
=====

```

The script installs the Python libraries 'shot-scraper', 'Playwright', and 'pillow' to capture live screenshots of webpages and apply a watermark containing the URL and timestamp.

- 'Playwright' is used to navigate to the target website from within the GitHub Actions container.
- 'shot-scraper' uses Playwright to launch a browser session and load the target URL. Once the page is fully rendered, it captures and saves a screenshot.
- 'Pillow' is used for post-processing the image. After the screenshot is captured, Pillow adds a small overlay in the bottom-left corner of the image containing the URL and timestamp.

The images are then included in a variable that will be used for Slack Image Blocks.

The two processed information for the billing and web health is constructed into tables for Slack Blocks.

```
# Create Tables for Slack Message
#=====
$BillingFields = @()
foreach ($Item in $ProcessedResources) {
    $NameString = '.' + $Item.Name + '.'
    $CostString = '+' + $Item.Cost + ' USD'

    $BillingFields += @(
        @{
            type = 'mrkdwn'
            text = $NameString
        }
        @{
            type = 'mrkdwn'
            text = $CostString
        }
    )

    if ($BillingFields.Count -eq 0) {
        $BillingFields += @(
            type = 'mrkdwn'; text = '_No active billing meters_'
        )
        $BillingFields += @(
            type = 'mrkdwn'; text = '_$0.00_'
        )
    }
}

$UrlFields = @()
foreach ($Row in $UrlTable) {
    $UrlString = '🌐 ' + $Row.'Target URL' + ' '

    $StatusMarker = '🟢 ' + $Row.'Status Code'
    if ($Row.'Status Code' -eq 200) {
        $StatusMarker = '🟢 200 OK'
    }
    $StatusString = '🟢 ' + $StatusMarker + '🟢'

    $UrlFields += @(
        type = 'mrkdwn'
        text = $UrlString
    )
    $UrlFields += @(
        type = 'mrkdwn'
        text = $StatusString
    )
}
#=====
```

```
# Construct Slack Body Message
#=====
$SlackBlocks = @(
    @{
        type = 'header'
        text = @{} type = 'plain_text'; text = 'Infrastructure Cost & Health Report'; emoji = $true
    },
    @{ type = 'divider' },
    @{
        type = 'section'
        text = @{} type = 'mrkdwn'; text = '"Month-to-Date PAYG Consumption"'
    },
    @{
        type = 'section'
        fields = $BillingFields
    },
    @{ type = 'divider' },
    @{
        type = 'section'
        text = @{} type = 'mrkdwn'; text = '"Static Web App Availability Manifest"'
    },
    @{
        type = 'section'
        fields = $UrlFields
    }
)

if ($SlackImageBlocks.Count -gt 0) {
    $SlackBlocks += @{} type = 'divider'
    $SlackImageBlocks += $SlackImageBlocks
}

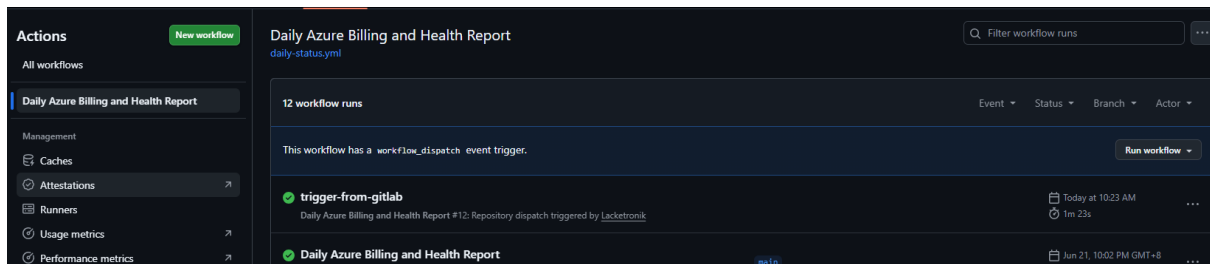
$Payload = @{} blocks = $SlackBlocks | ConvertTo-Json -Depth 10 -Compress
#=====
```

The Slack Block message is dynamically constructed to display the two tables and screenshots in a clean, easy-to-read format. Once assembled, the payload is converted into JSON and out-filed into so that GitHub Action can send it to Slack through an incoming webhook for delivery once it has committed the images.

```
# Write Payload out so that Slack can pick it up after Git Commit the images
#=====
$Payload | Out-File -FilePath "slack_payload.json" -Encoding utf8
Write-Host "Payload cached locally. Ready for Git deployment!"
#=====
```

GitHub Action

A workflow YAML script is created alongside a GitHub Action that would spin up a virtual linux machine that will checkout the repo, install the relevant dependencies, execute the Powershell script, commit the images to the repo and send the Slack message.



In the final three steps of the workflow, the action will:

- Execute the PowerShell script using the secret Azure Subscription ID.
- Commit and push the screenshots to the repository.
- Invoke the Slack Webhook API to deliver the notification message using the secret Slack Webhook URL

```
- name: Run Billing Parsing & Screenshot Script
  shell: pwsh
  run: |
    ./Query_Azure.ps1
  env:
    AZURE_SUBSCRIPTION_ID: ${ secrets.AZURE_SUBSCRIPTION_ID }

- name: Commit and Push Latest Screenshots
  run: |
    git config --global user.name "github-actions[bot]"
    git config --global user.email "github-actions[bot]@users.noreply.github.com"

    git add docs/*.png

    git commit -m "Update daily live web snapshots [skip ci]" || exit 0
    git push

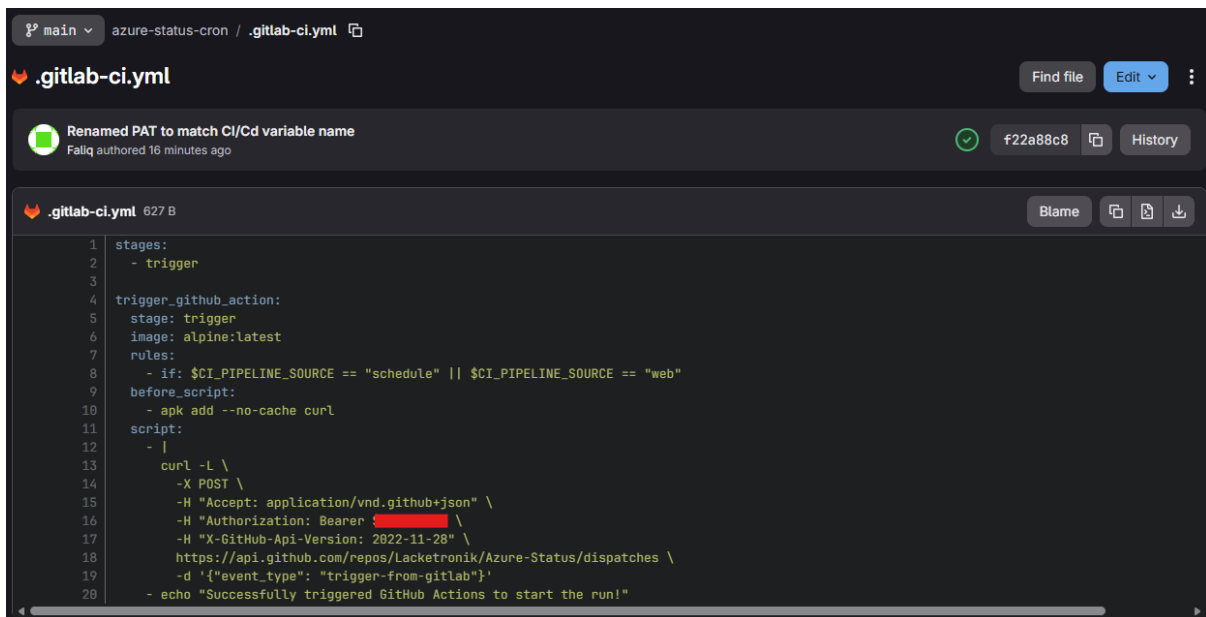
    sleep 5

- name: Post Final Report to Slack
  shell: pwsh
  run: |
    $Payload = Get-Content -Raw -Path "slack_payload.json"
    Invoke-RestMethod -Method Post -Uri "${ secrets.SLACK_WEBHOOK_URL }" -Body $Payload -ContentType "application/json; charset=utf-8"
```

GitLab Schedule

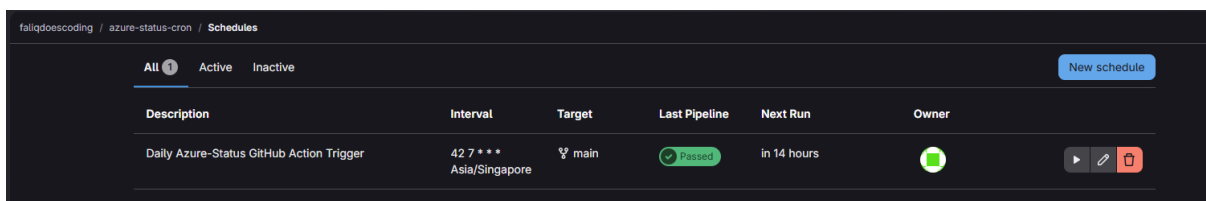
Originally, the GitHub Action used a cron schedule to run daily at a predefined time. However, the schedule proved unreliable, with the workflow not running as expected on multiple occasions.

As a workaround, an external scheduler was implemented to trigger the workflow via an HTTP POST request. A lightweight GitLab project was created containing a single YAML pipeline that calls the GitHub Actions workflow dispatch API. Authentication is handled using a GitHub Personal Access Token (PAT).



```
1 stages:
2   - trigger
3
4 trigger_github_action:
5   stage: trigger
6   image: alpine:latest
7   rules:
8     - if: $CI_PIPELINE_SOURCE == "schedule" || $CI_PIPELINE_SOURCE == "web"
9   before_script:
10    - apk add --no-cache curl
11   script:
12     - |
13       curl -L \
14         -X POST \
15         -H "Accept: application/vnd.github+json" \
16         -H "Authorization: Bearer [REDACTED]" \
17         -H "X-GitHub-API-Version: 2022-11-28" \
18         https://api.github.com/repos/Lacketronik/Azure-Status/dispatches \
19         -d '{"event_type": "trigger-from-gitlab"}'
20     - echo "Successfully triggered GitHub Actions to start the run!"
```

A pipeline is scheduled to run daily at 07:42 am (SGT).



Description	Interval	Target	Last Pipeline	Next Run	Owner
Daily Azure-Status GitHub Action Trigger	42 7 * * * Asia/Singapore	main	Passed	in 14 hours	

Slack Message

The crafted Slack Block message is then delivered to the azure-subscription channel through a Webhook. Since the workflow relies on public GitLab pipelines, occasional delays in job execution may occur.

The message provides Month-to-Date (MTD) cost information for each Azure resource, Web App availability status based on HTTP response codes and live webpage screenshots to facilitate visual monitoring and validation.

faliqdoescoding Get 50% off Slack Pro

azure-subscription

Azure-Subscription APP 8:15 AM

Infrastructure Cost & Health Report

Month-to-Date PAYG Consumption

Resource	Cost
defaultworkspace-	0.03212 USD
cosmos-myresume	0 USD
stmyresume	0.00188 USD
func-myresume	0.00042 USD

Static Web App Availability

URL	Status
https://purple-coast-065cac210.7.azurestaticapps.net	200 OK
https://faliqdoescoding.com	200 OK
https://www.faliqdoescoding.com	200 OK

Visual Check: purple-coast-065cac210.7.azurestaticapps.net (229 kB)

This web resume is continuously being worked on! Email or connect with me for suggestions or feedback.

Full-Stack Developer | Cloud & DevOps Enthusiast

Projects Experience Education Certification

All Tags

MyResume - Modern Cloud-native Portfolio

Azure Terraform React TypeScript C# .NET Core SQL Linux

Bots TD - 3D First-person Tower Defense Game

C++ OpenGL

SingaPourCast - Scalable Weather Forecasting Cloud Infrastructure

AWS PHP HTML CSS

Driving Sim - Android Game Featuring Gyroscopic-based Vehicle Control

https://www.faliqdoescoding.com

Captured 2020-06-23 08:15:50 LT

This web resume is continuously being worked on! Email or connect with me for suggestions or feedback.

Full-Stack Developer | Cloud & DevOps Enthusiast

Projects Experience Education Certification

All Tags

MyResume - Modern Cloud-native Portfolio

Azure Terraform React TypeScript C# .NET Core SQL Linux

Bots TD - 3D First-person Tower Defense Game

C++ OpenGL

SingaPourCast - Scalable Weather Forecasting Cloud Infrastructure

AWS PHP HTML CSS

Driving Sim - Android Game Featuring Gyroscopic-based Vehicle Control

https://www.faliqdoescoding.com

Captured 2020-06-23 08:15:50 LT

(Notice the watermark, site URL and timestamp; 0815 am when scheduled 0742 am)

Conclusion

Azure-Status is an automated monitoring and reporting solution that provides daily visibility into Azure costs and web application availability. By combining Azure REST APIs, PowerShell, GitHub Actions, GitLab CI/C and Slack Block Kit, the workflow eliminates manual reporting and delivers a consolidated operational summary directly to Slack.

The project demonstrates how multiple tools and services can be integrated to create a reliable, end-to-end automation pipeline for cloud monitoring and health validation reporting.